



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems Lecture 15

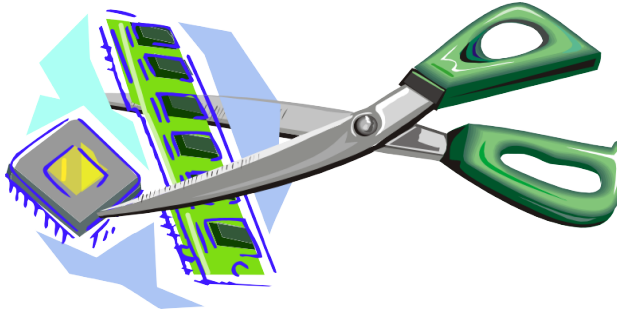
Memory: virtual memory, segments and page tables

**Prof. Yunming XIAO
School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

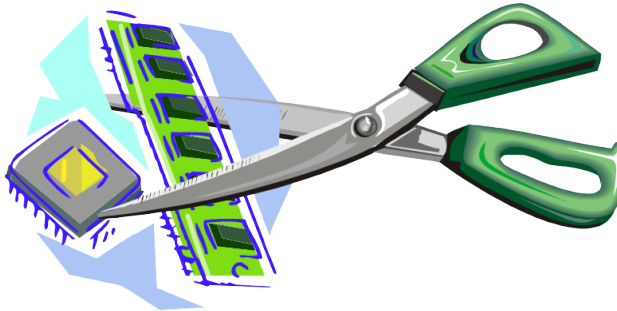
Recall: memory basics

Virtualizing resources



- Physical reality: different processes/threads share the same hardware
 - Need to multiplex CPU (just finished: scheduling)
 - Need to multiplex use of memory (starting today)
 - Need to multiplex disk and devices (later in term)

Virtualizing resources



- Why worry about memory sharing?
 - The complete working state of a process and/or kernel is defined by its data in memory (and registers)
 - Consequently, cannot just let different threads of control use the same memory
 - Physics: two different pieces of data cannot occupy the same locations in memory
 - Probably don't want different threads to even have access to each other's memory if in different processes (protection)

Important aspects of memory multiplexing

- **Protection:**

- Prevent access to private memory of other processes
 - Different pages of memory can be given special behavior (read only, invisible to user programs, etc).
 - Kernel data protected from User programs
 - Programs protected from themselves

- **Translation:**

- Ability to translate accesses from one address space (virtual) to a different one (physical)
- When translation exists, process uses virtual addresses
- Side effects:
 - Can be used to avoid overlap
 - Can be used to give uniform view of memory to programs

Important aspects of memory multiplexing

- **Controlled overlap:**
 - Separate state of threads should not collide in physical memory. Obviously, unexpected overlap causes chaos!
 - Conversely, would like the ability to overlap when desired (for communication)

Alternative view: interposing on process behavior

- OS interposes on process' I/O operations
 - How? All I/O happens via syscalls.
- OS interposes on process' CPU usage
 - How? Interrupt lets OS preempt current thread
- **Question: how can the OS interpose on process' memory accesses?**
 - Too slow for the OS to interpose every memory access
 - Translation: hardware support to accelerate the common case
 - Page fault: uncommon cases trap to the OS to handle

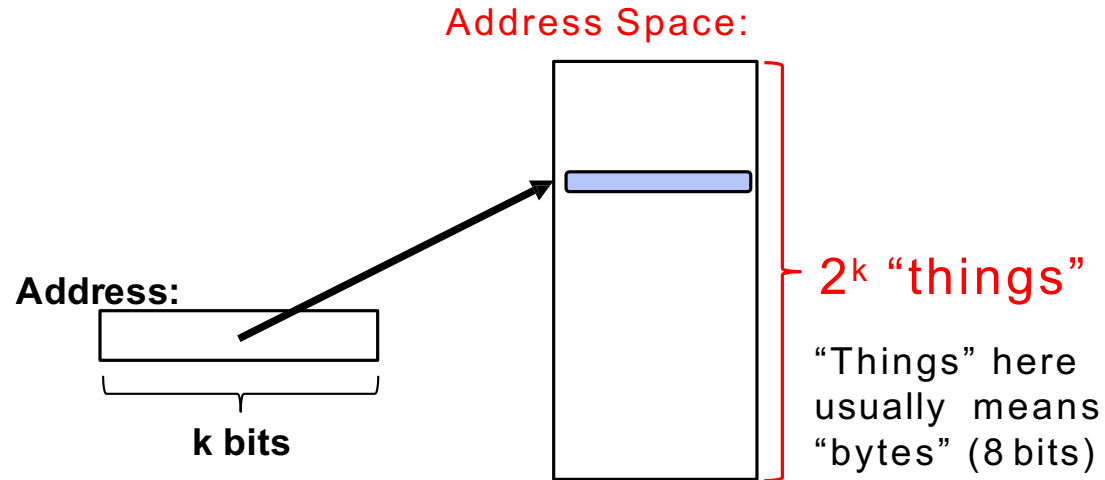
Recall: four fundamental OS concepts

- Thread: Execution Context
 - Fully describes program state
 - Program Counter, Registers, Execution Flags, Stack
- **Address space (with or w/o translation)**
 - Set of memory addresses accessible to program (for read or write)
 - May be distinct from memory space of the physical machine (in which case programs operate in a virtual address space)

Recall: four fundamental OS concepts

- Process: an instance of a running program
 - Protected Address Space + one or more threads
- **Dual mode operation / Protection**
 - Only the “system” can access certain resources
 - Combined with translation, isolates user processes from each other and the OS from user processes

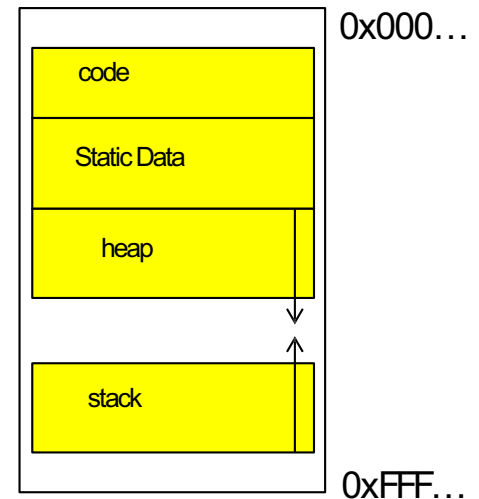
THE BASICS: address/address space



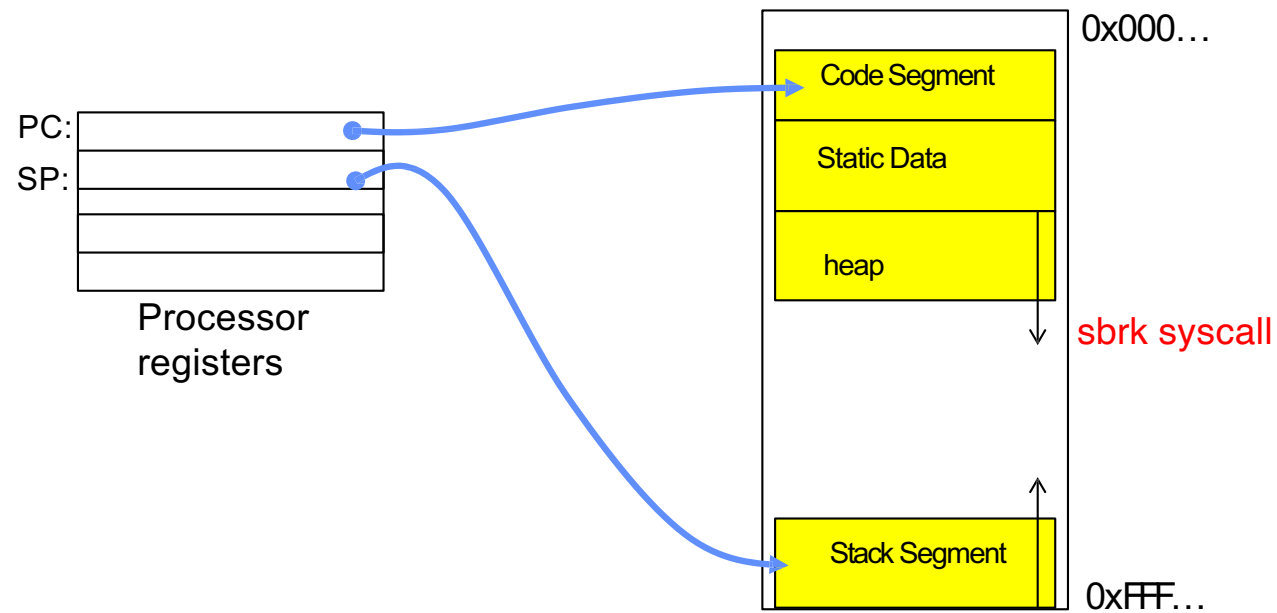
- What is 210 bytes (where a byte is abbreviated as “B”)?
 - $2^{10} \text{ B} = 1024\text{B} = 1 \text{ KB}$ (for memory, 1K = 1024, not 1000)
- How many bits to address each byte of 4KB page?
 - $4\text{KB} = 4 \times 1\text{KB} = 4 \times 2^{10} = 2^{12} \Rightarrow 12 \text{ bits}$
- How much memory can be addressed with 20 bits? 32 bits? 64 bits?
 - Use 2^k

Process (virtual) address space

- Definition: set of accessible addresses and the state associated with them
 - $2^{32} = \sim 4$ billion bytes on a 32-bit machine
- How many 32-bit numbers fit in this address space?
 - 32-bits = 4 bytes, so $2^{32}/4 = 2^{30} = \sim 1$ billion
- What happens when processor reads or writes to an address?
 - Perhaps acts like regular memory
 - Perhaps causes I/O operation
 - (Memory-mapped I/O)
 - Causes program to abort (segfault)?
 - Communicate with another program
 - ...

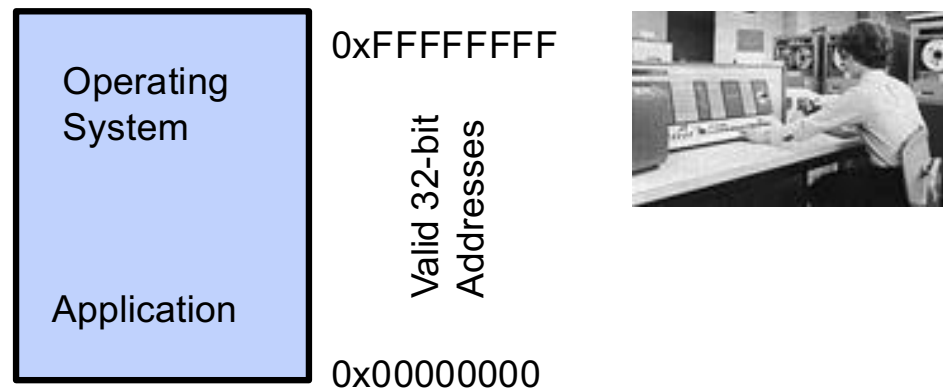


Recall: process address space: typical structure



Uniprogramming

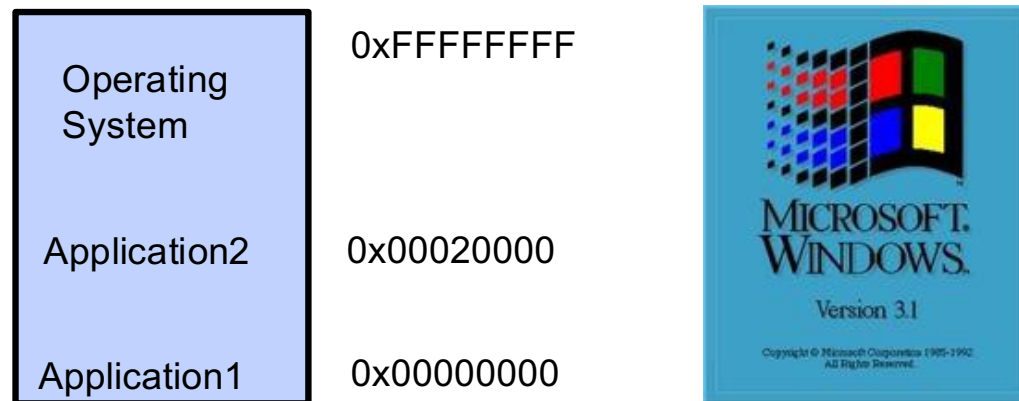
- Uniprogramming (no translation or protection)
 - Application always runs at same place in physical memory since only one application at a time
 - Application can access any physical address



- Application given illusion of dedicated machine by giving it reality of a dedicated machine

Primitive multiprocessing

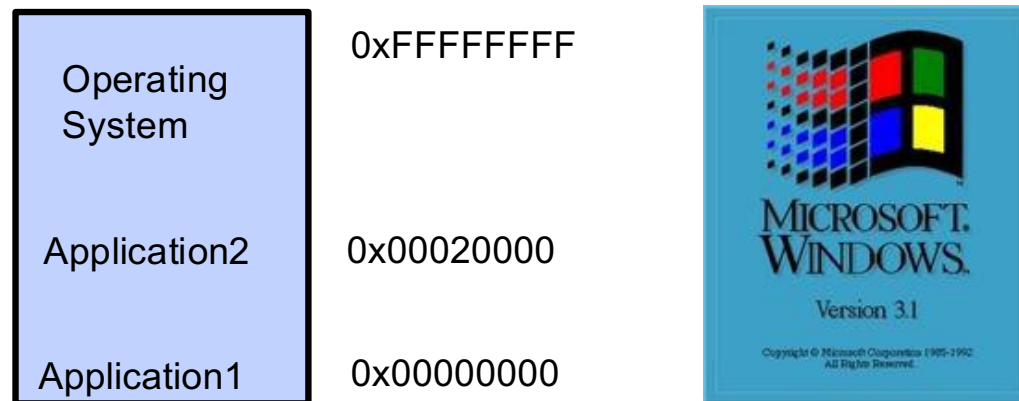
- Multiprogramming without translation or protection
 - Must somehow prevent address overlap between threads



- Use loader/linker: adjust addresses while program loaded into memory (loads, stores, jumps)
- Everything adjusted to memory location of program
- Translation done by a linker-loader (relocation)
- Common in early days (... till Windows 3.x, 95?)

Primitive multiprocessing

- Multiprogramming without translation or protection
 - Must somehow prevent address overlap between threads



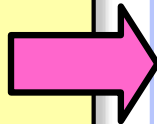
- With this solution, no protection: bugs in any program can cause other programs to crash or even the OS

Recall: memory protection

Binding of instructions and data to memory

Process view of memory

```
data1:  dw    32
        ...
start:  lw     r1,0(data1)
        jal   checkit
loop:   addi   r1, r1, -1
        bnz   r1, loop
        ...
checkit: ...
```



Physical Addresses

```
0x0300  00000020
...
0x0900  8C2000C0
0x0904  0C000280
0x0908  2021FFFF
0x090C  14200242
...
0x0A00
```

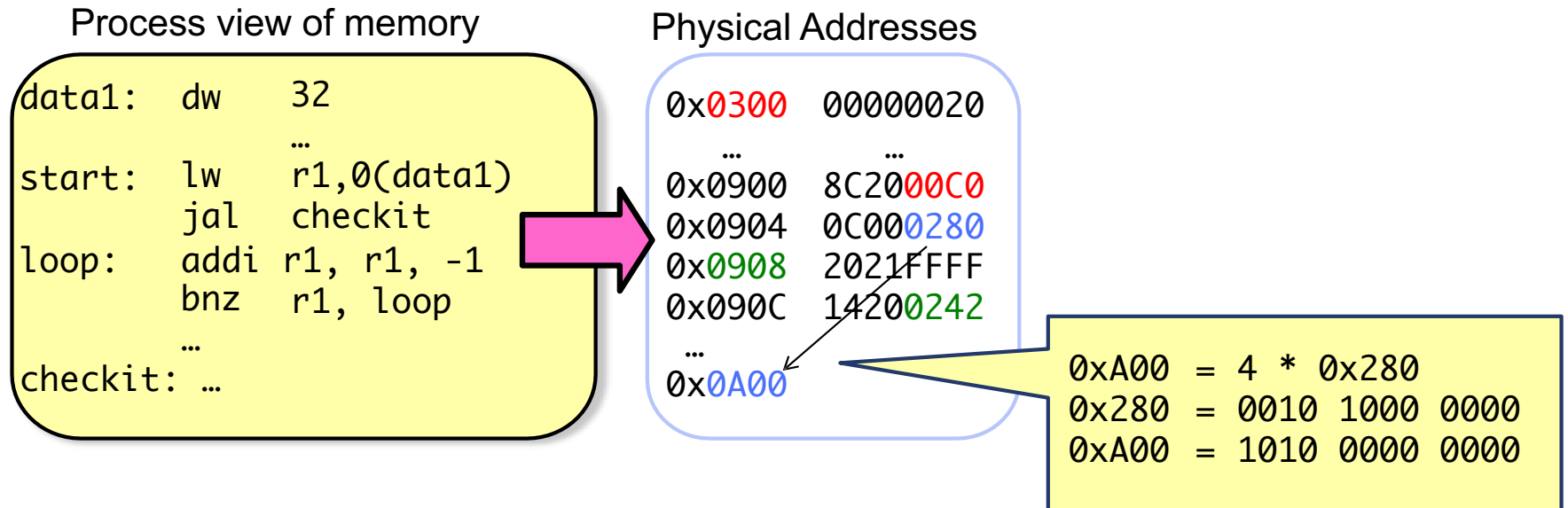
Assume 4byte words

$0x300 = 4 * 0x0C0$

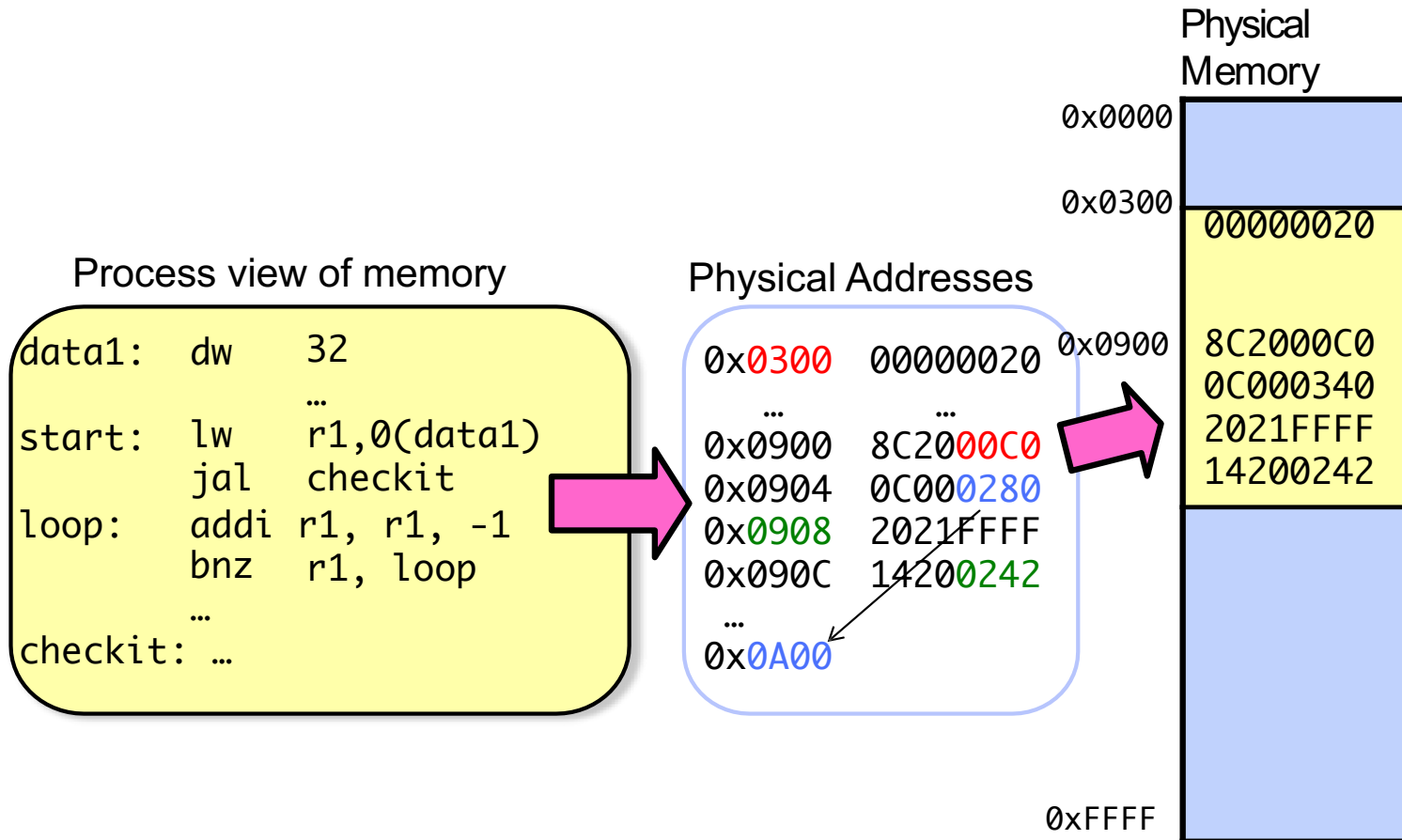
$0x0C0 = 0000\ 1100\ 0000$

$0x300 = 0011\ 0000\ 0000$

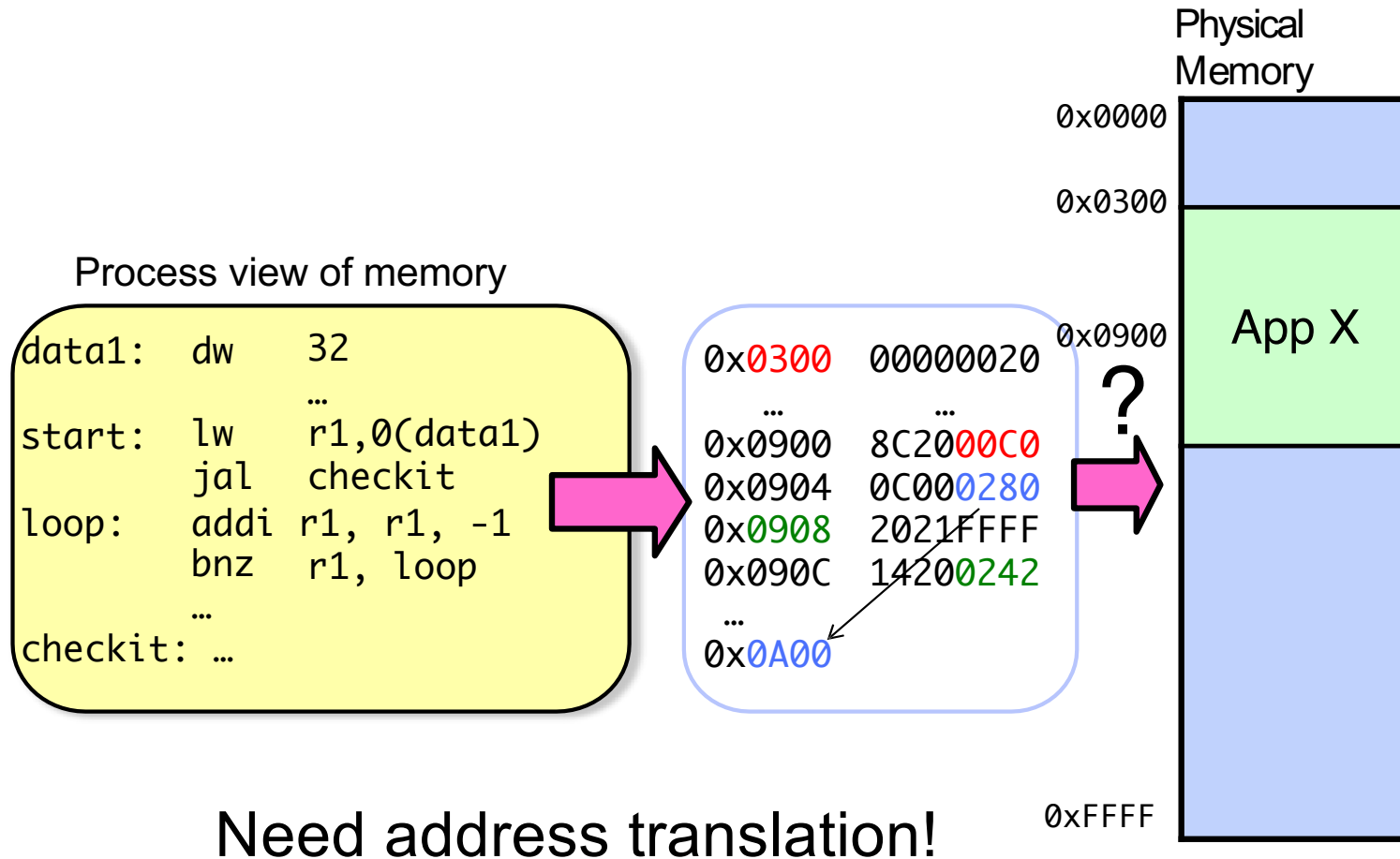
Binding of instructions and data to memory



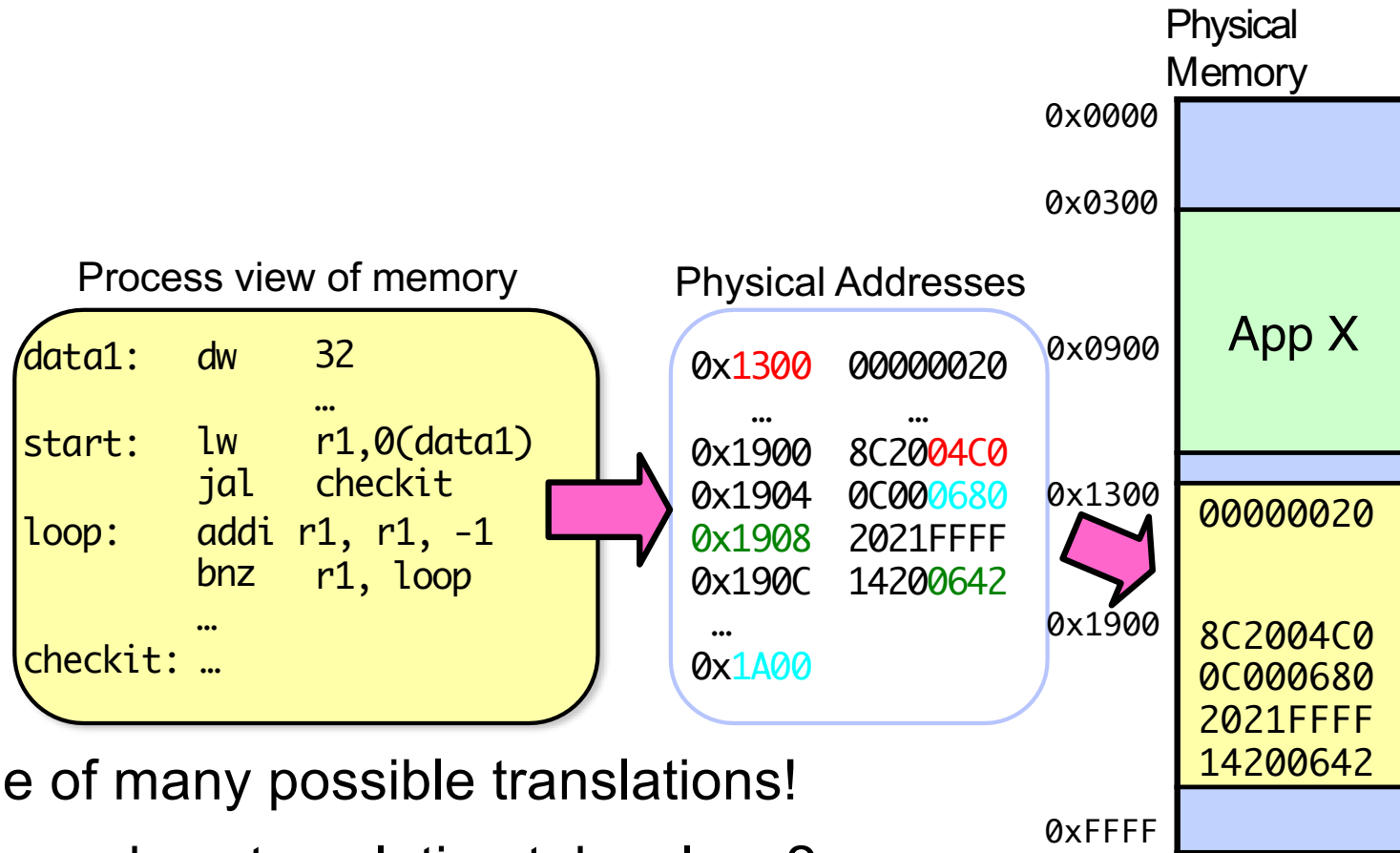
Binding of instructions and data to memory



Second copy of program from previous example



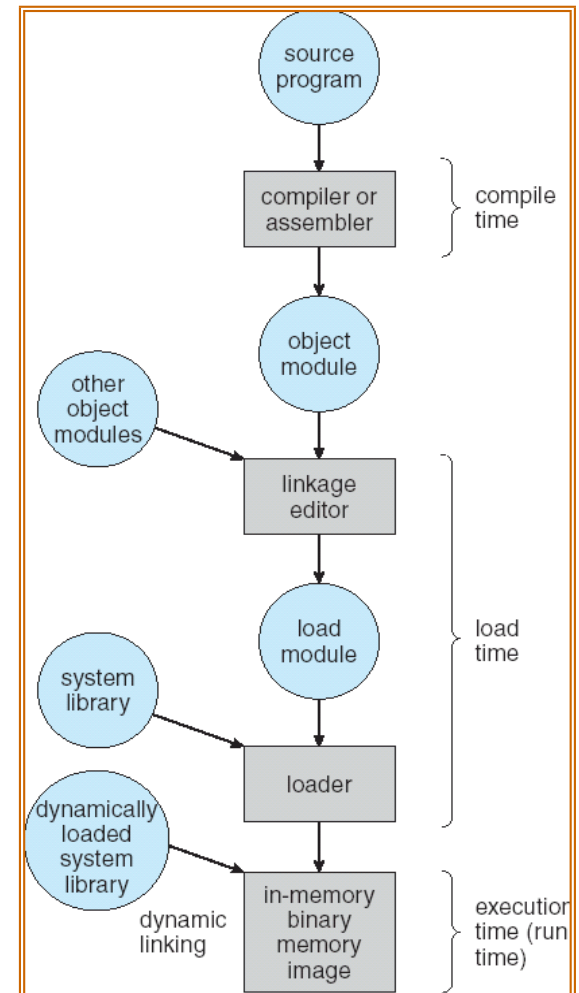
Second copy of program from previous example



- One of many possible translations!
- Where does translation take place?
 - Compile time, link/load time, or execution time?

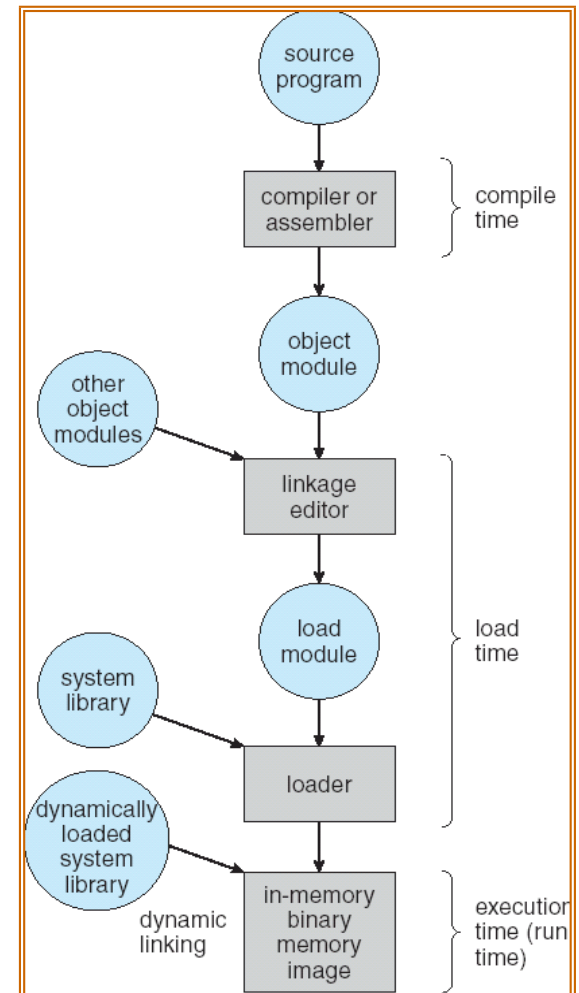
From program to process

- Preparation of a program for execution involves components at:
 - Compile time (i.e., “gcc”)
 - Link/Load time (UNIX “ld” does link)
 - Execution time (e.g., dynamic libs)
- Addresses can be bound to final values anywhere in this path
 - Depends on hardware support
 - Also depends on operating system



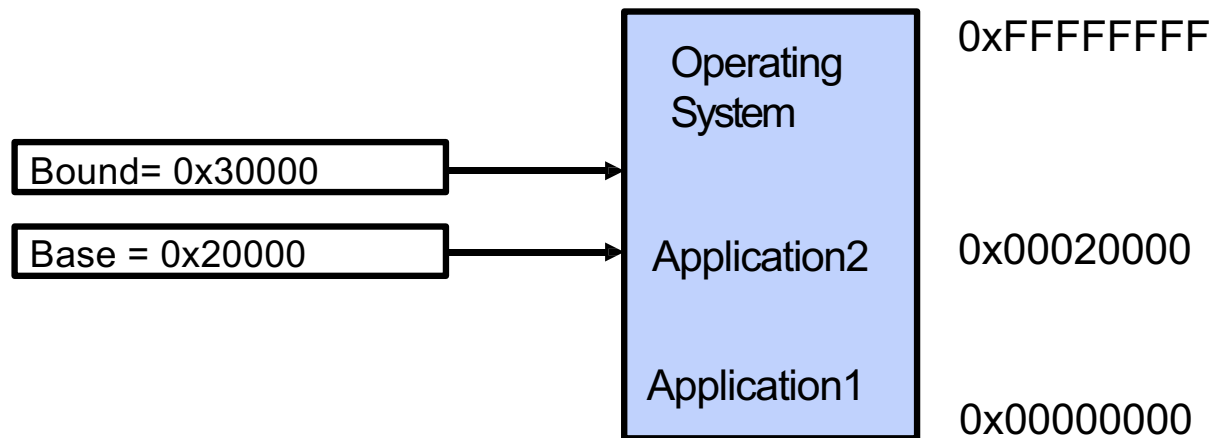
From program to process

- Dynamic libraries
 - Linking postponed until execution
 - Small piece of code (i.e. the stub), locates appropriate memory-resident library routine
 - Stub replaces itself with the address of the routine, and executes routine



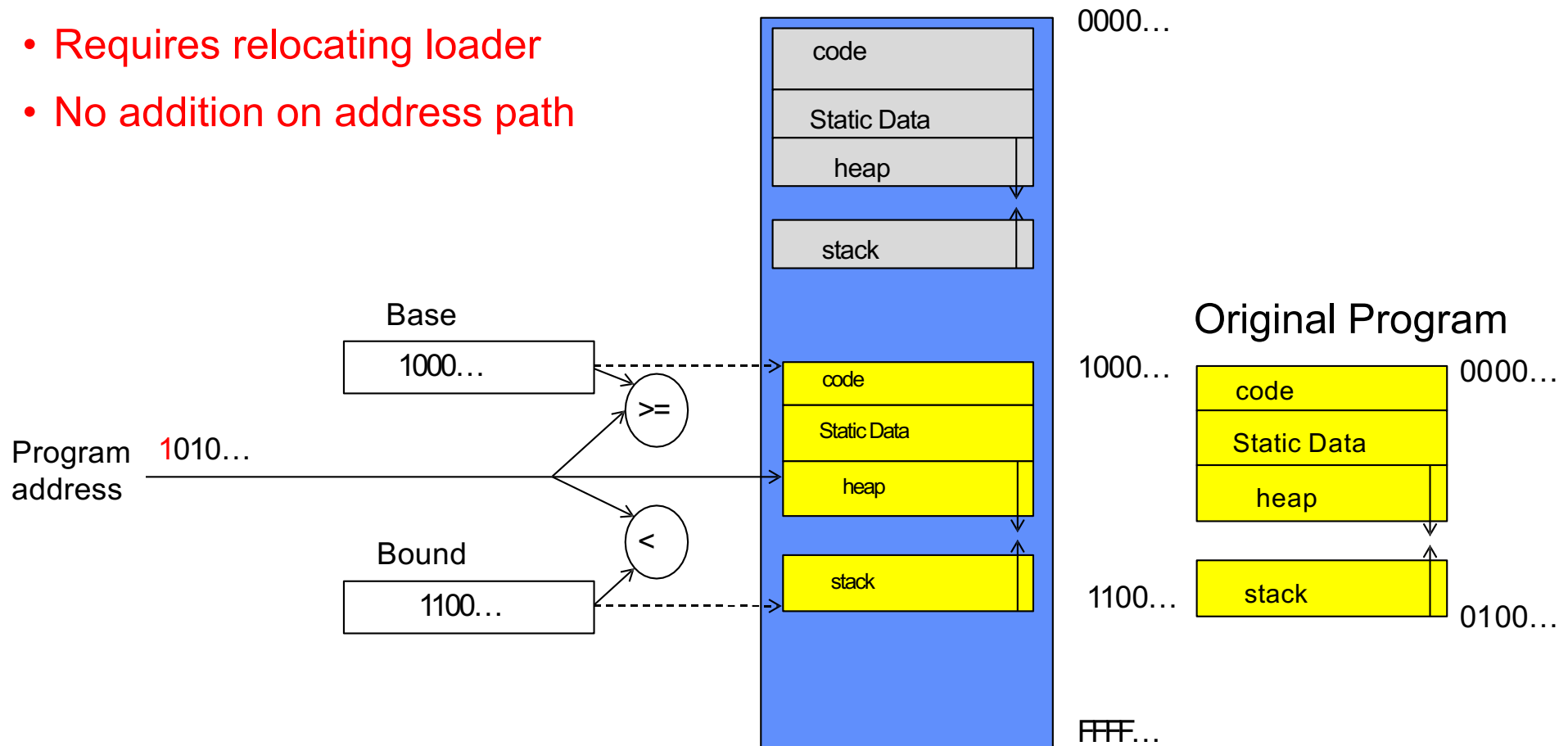
Multiprogramming with protection

- Can we protect programs from each other without translation?
 - Yes: Base and Bound!
 - Used by, e.g., Cray-1 supercomputer

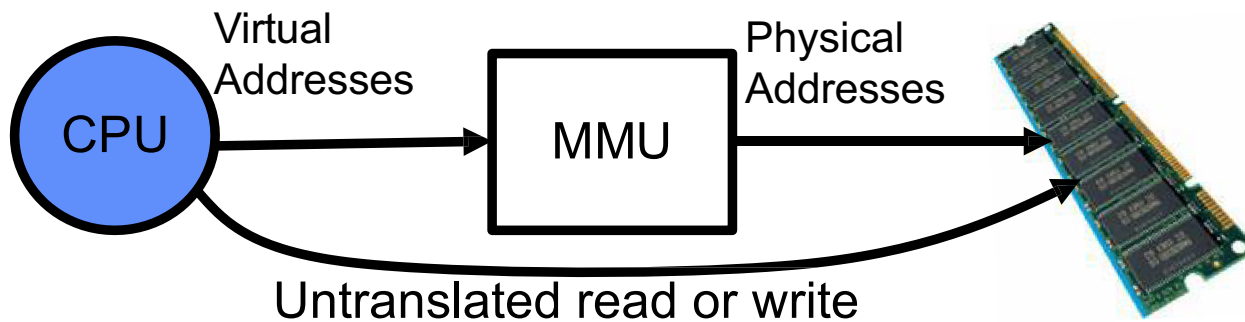


Recall: base and bound (no translation)

- Still protects OS and isolates program
- Requires relocating loader
- No addition on address path

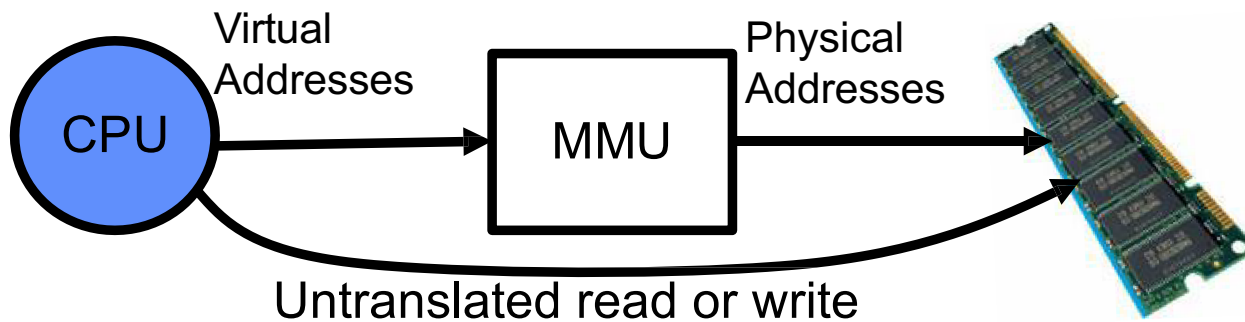


Recall: general address translation



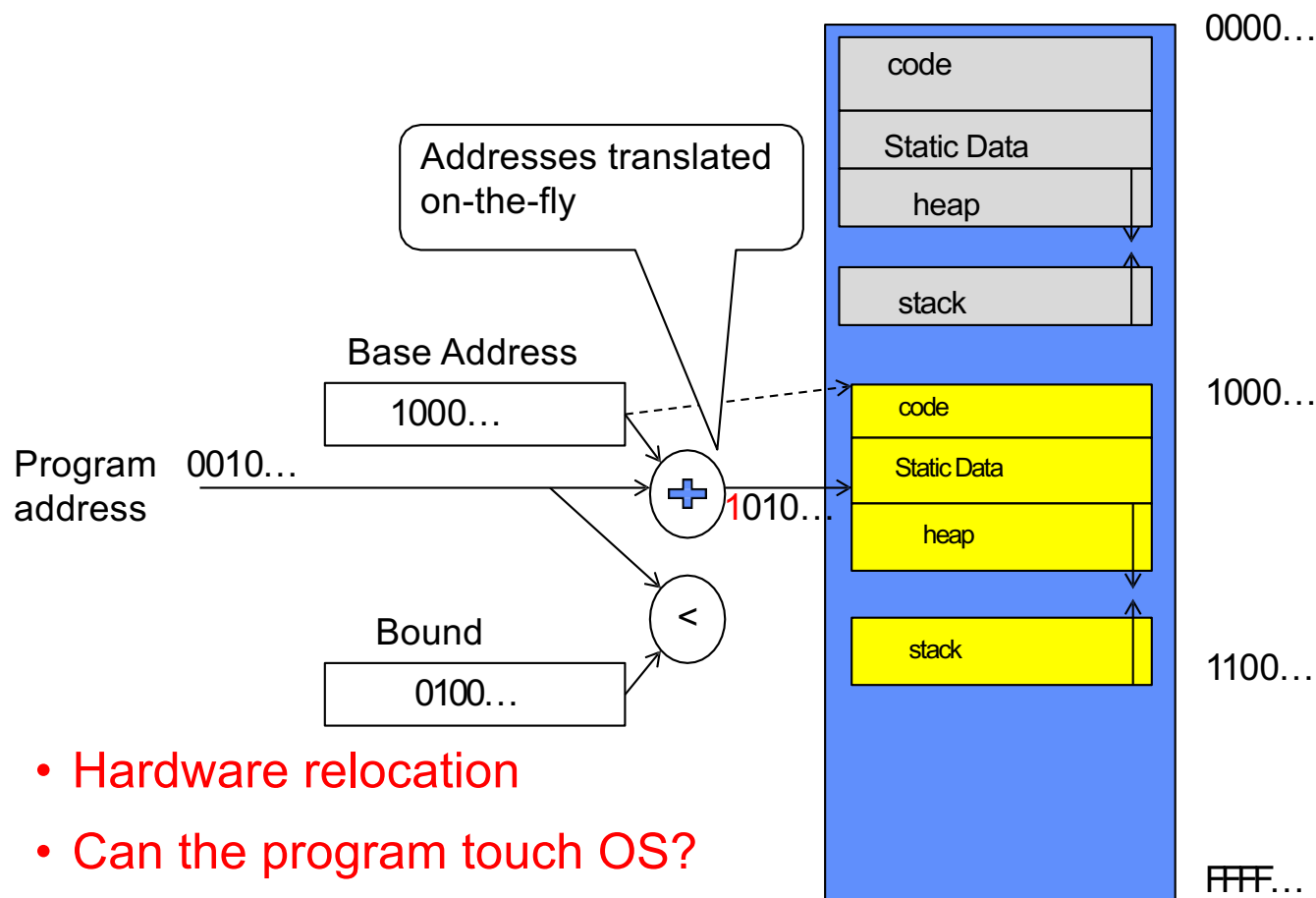
- Consequently, two views of memory:
 - View from the CPU (what program sees, virtual memory)
 - View from memory (physical memory)
 - **Translation box** (Memory Management Unit or MMU) converts between two views

Recall: general address translation

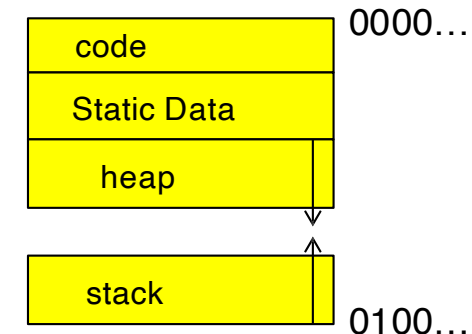


- Translation $\Rightarrow$ much easier to implement protection!
 - If task A cannot even gain access to task B's data, no way for A to adversely affect B
- With translation, every program can be linked/loaded into same region of user address space

Recall: base and bound (with translation)

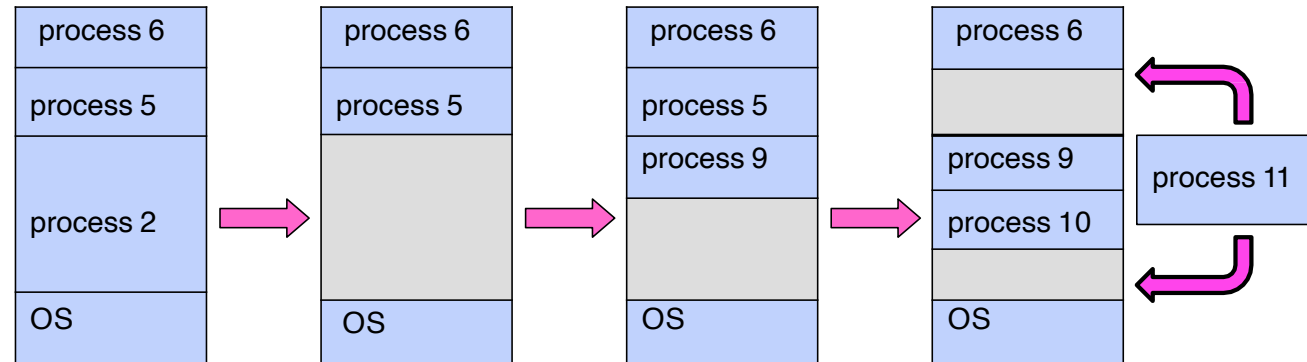


Original Program



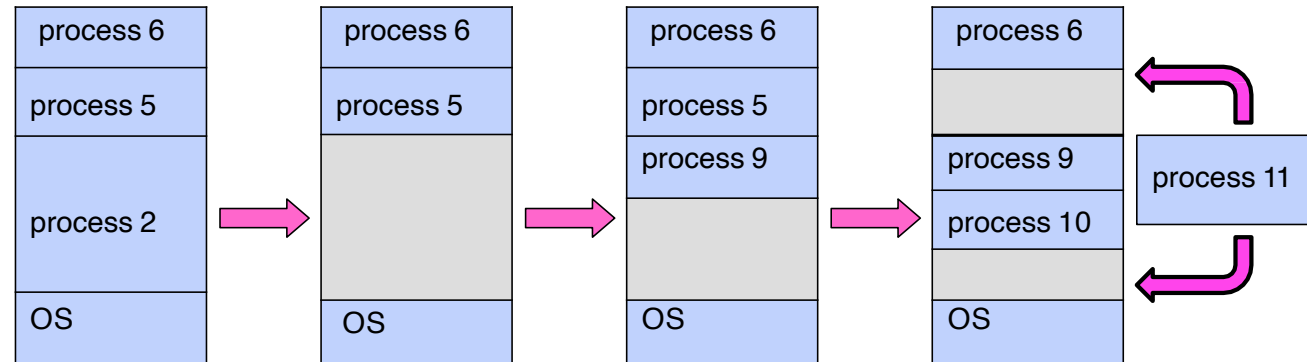
- Hardware relocation
- Can the program touch OS?
- Can it touch other programs?

Issues with simple B&B method



- Fragmentation problem over time
 - Not every process is same size $\Rightarrow$ memory becomes fragmented over time
 - Fragmentation: wasted space both *external* (between blocks) and *internal* (inside blocks)

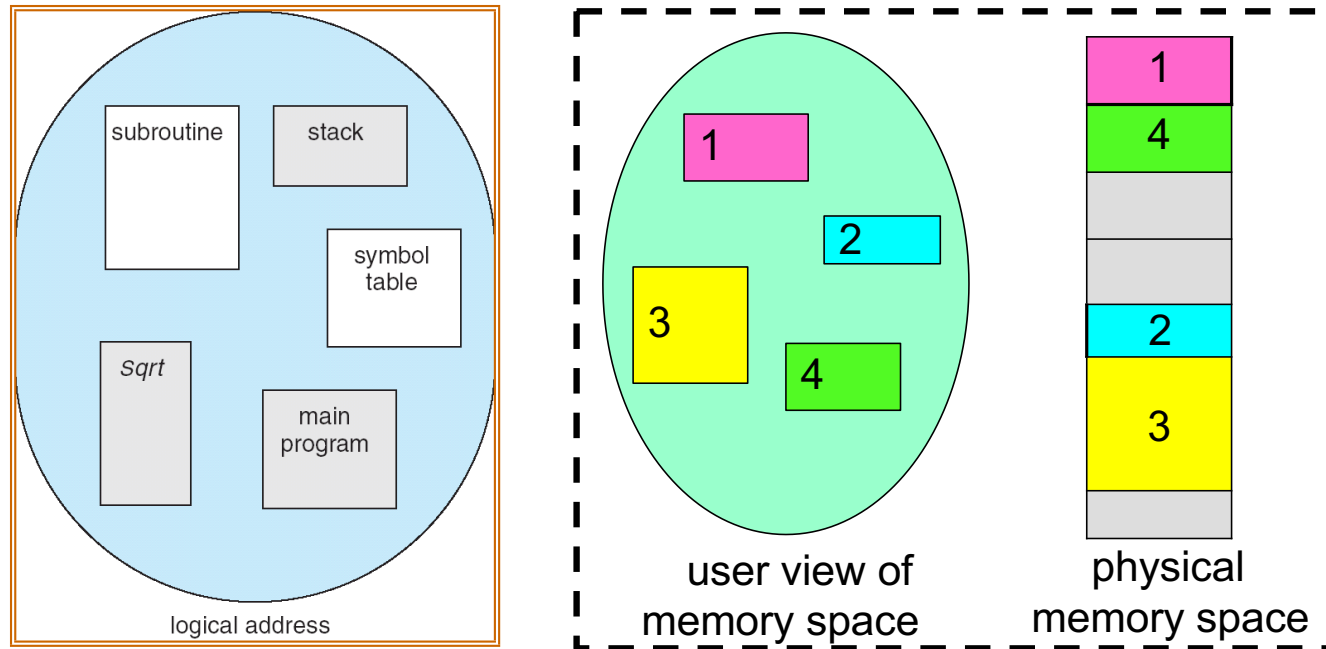
Issues with simple B&B method



- Missing support for sparse address space
 - Would like to have multiple chunks/program (code, data, stack, heap, etc)
- Hard to do inter-process sharing
 - Want to share code segments when possible
 - Want to share memory between processes
 - Helped by providing multiple segments per process

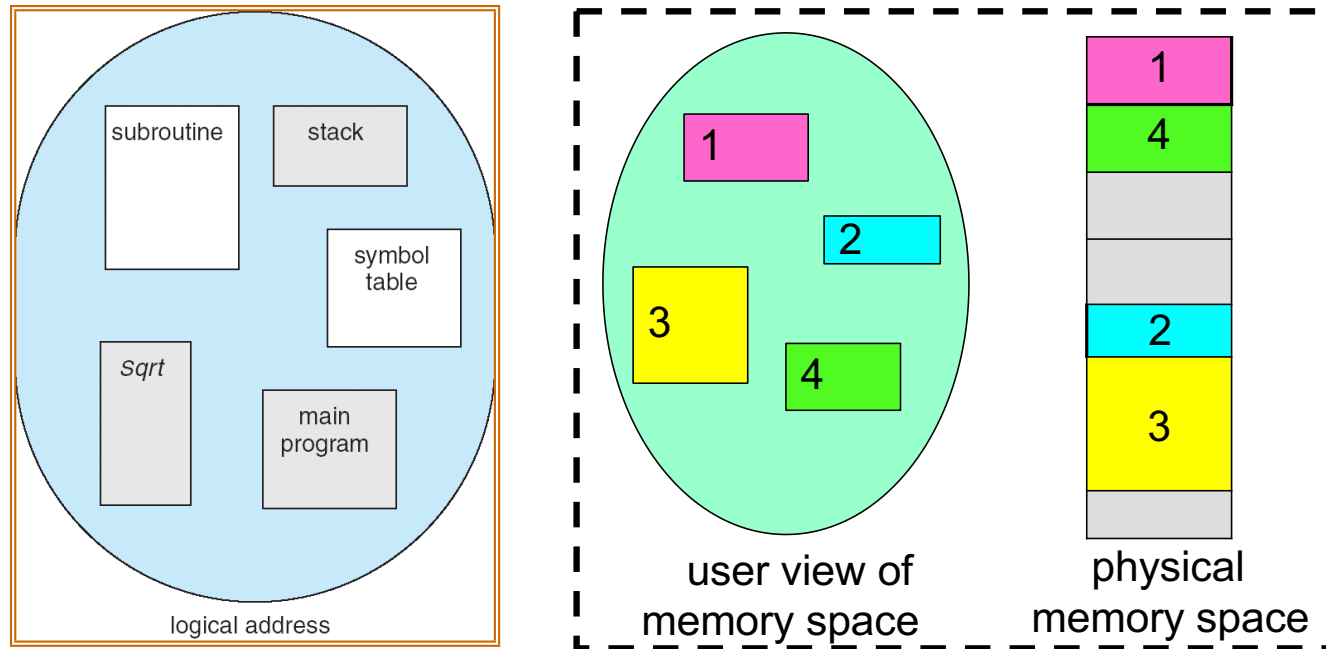
Segmentation

More flexible segmentation



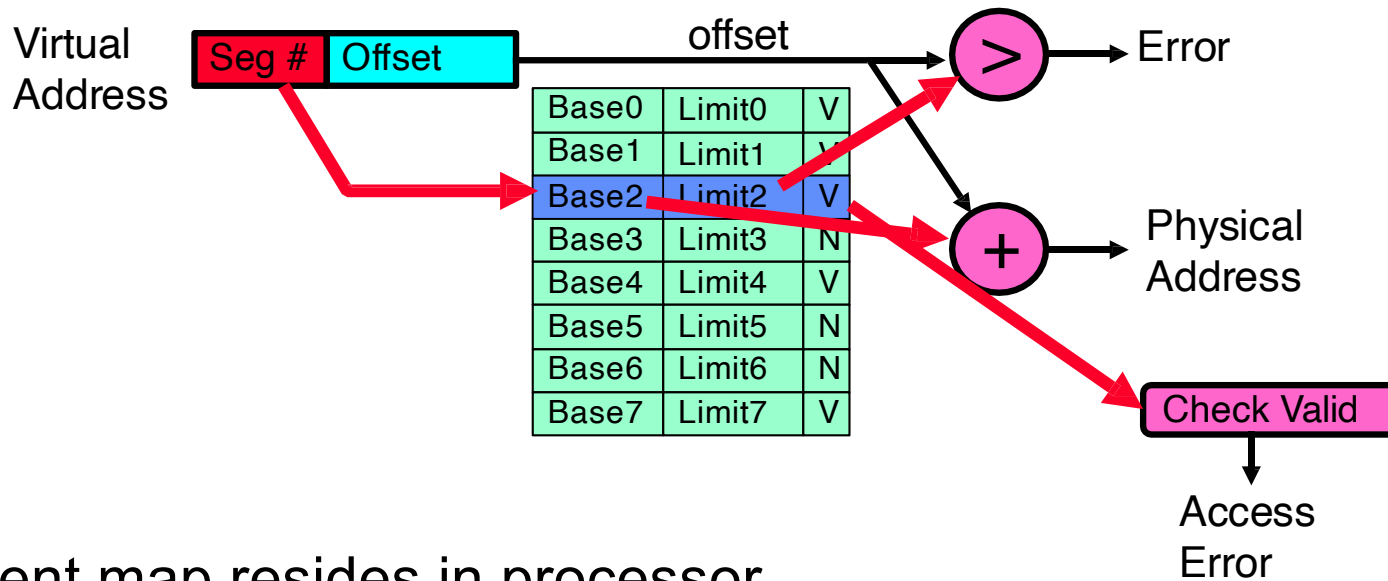
- Logical view: multiple separate segments
 - Typical: Code, Data, Stack
 - Others: memory sharing, etc

More flexible segmentation



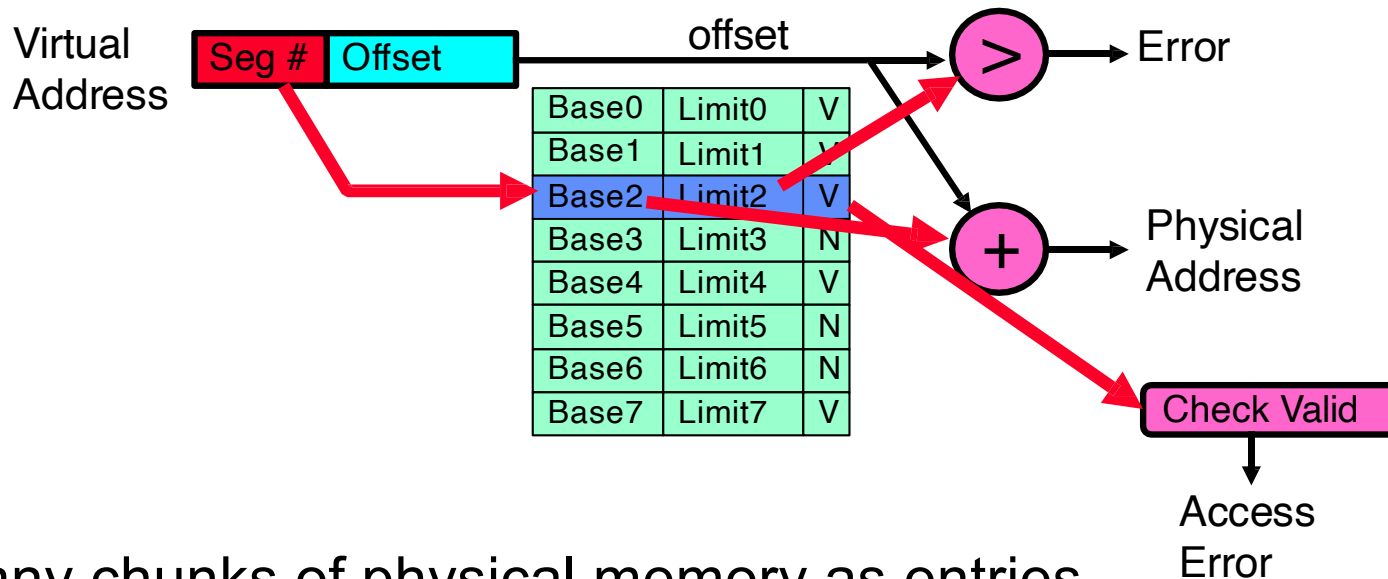
- Each segment is given region of contiguous memory
 - Has a base and limit
 - Can reside anywhere in physical memory

Implementation of multi-segment model



- Segment map resides in processor
 - Segment number mapped into base/limit pair
 - Base added to offset to generate physical address
 - Error check catches offset out of range

Implementation of multi-segment model



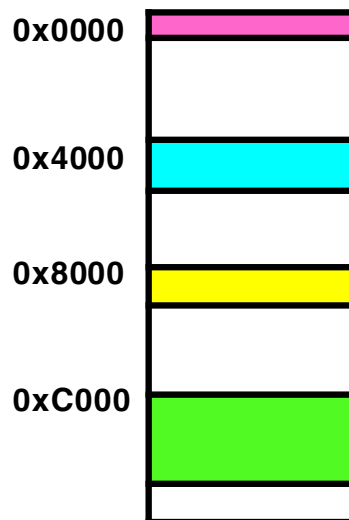
- As many chunks of physical memory as entries
 - Segment addressed by portion of virtual address
 - However, could be included in instruction instead:
 - x86 Example: `mov [es:bx], ax`
- What is “V/N” (valid / not valid)?
 - Can mark segments as invalid; requires check as well

Example: four segments (16 bit addresses)

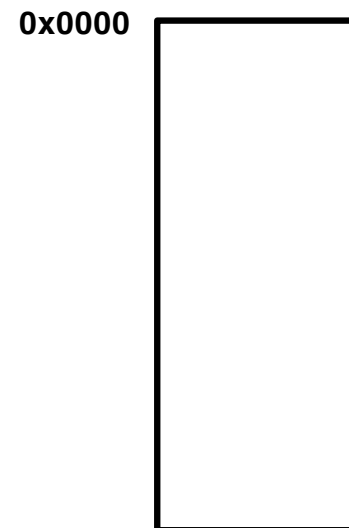


Virtual Address Format

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000



Virtual
Address Space



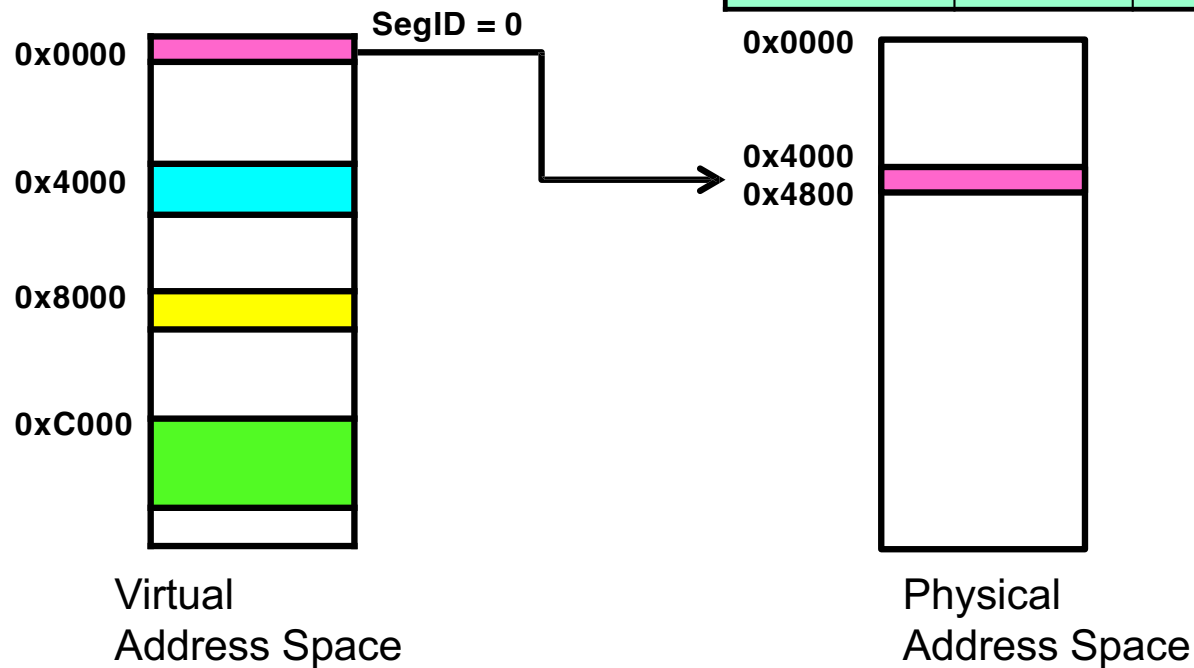
Physical
Address Space

Example: four segments (16 bit addresses)



Virtual Address Format

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

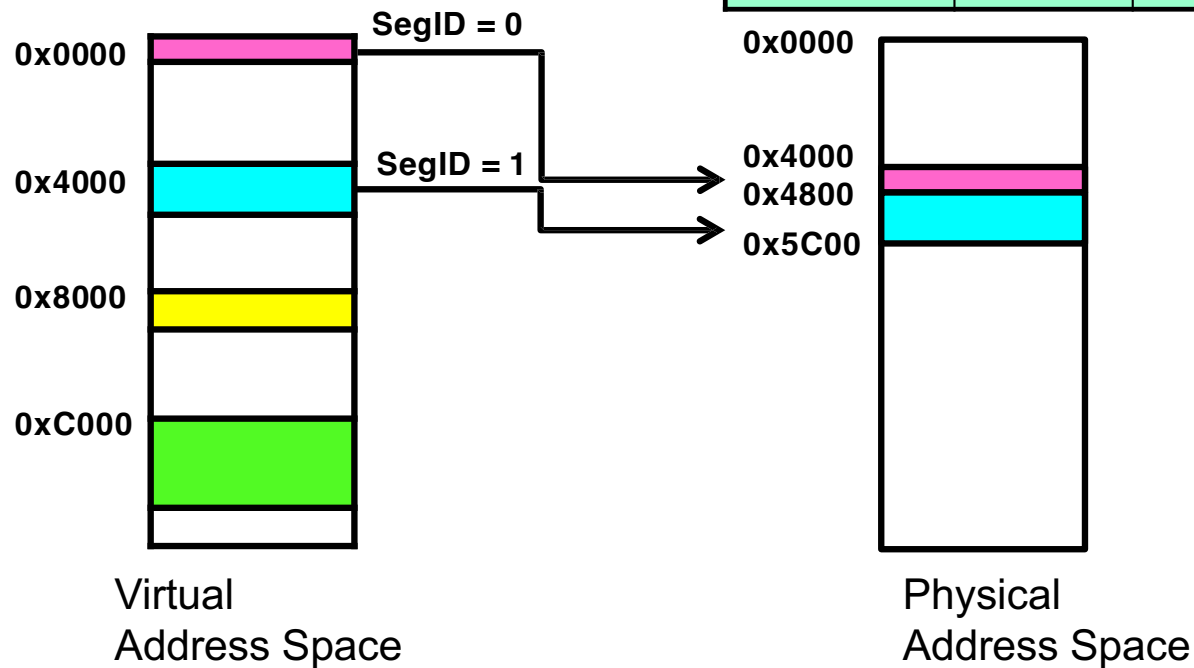


Example: four segments (16 bit addresses)



Virtual Address Format

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

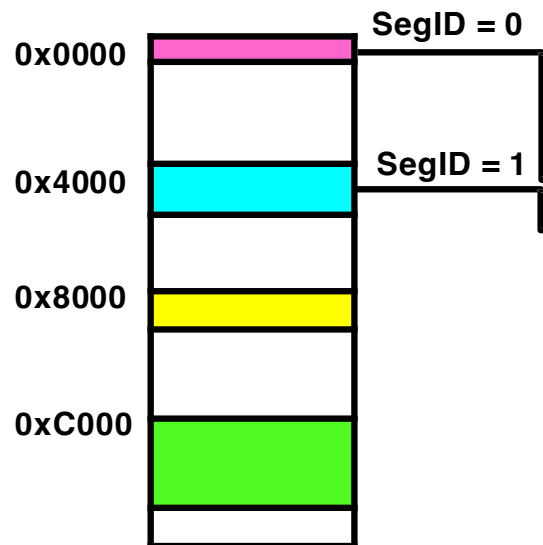


Example: four segments (16 bit addresses)

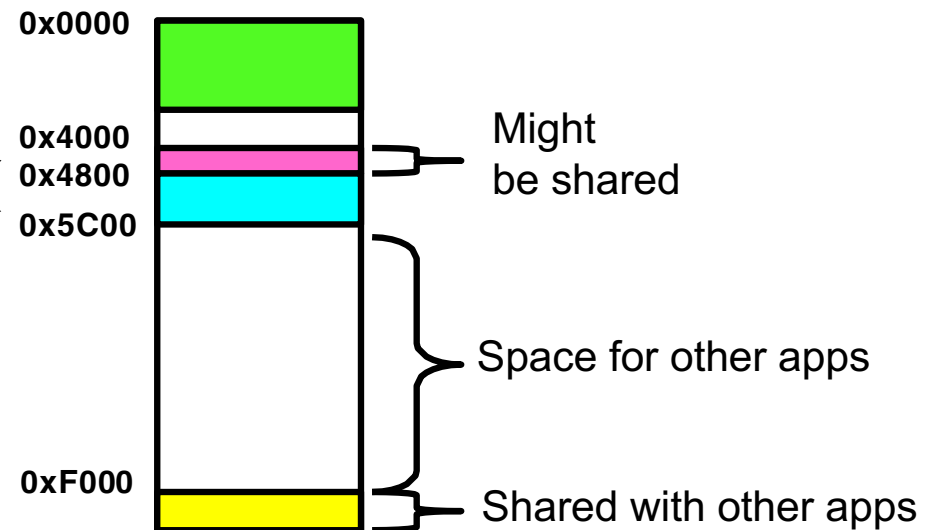


Virtual Address Format

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000



Virtual Address Space



Physical Address Space

Example of segment translation (16bit address)

```
0x0240  main:    la $a0, varx
0x0244                jal strlen
...
0x0360  strlen:  li    $v0, 0 ;count
0x0364  loop:    lb    $t0, ($a0)
0x0368                beq $r0,$t0, done
...
0x4050  varx     dw    0x314159
```

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

Let's simulate a bit of this code to see what happens (PC=0x0240):

1. Fetch 0x0240 (0000 0010 0100 0000).

Virtual segment #? 0; Offset? 0x240

Physical address? Base=0x4000, so physical addr=0x4240

Fetch instruction at 0x4240. Get "la \$a0, varx"

Move 0x4050 → \$a0, Move PC+4→PC

Example of segment translation (16bit address)

0x0240	main:	la	\$a0, varx
0x0244		jal	strlen
...			...
0x0360	strlen:	li	\$v0, 0 ;count
0x0364	loop:	lb	\$t0, (\$a0)
0x0368		beq	\$r0,\$t0, done
...			...
0x4050	varx	dw	0x314159

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

Let's simulate a bit of this code to see what happens (PC=0x0240):

1. Fetch 0x0240 (0000 0010 0100 0000).

Virtual segment #? 0; Offset? 0x240

Physical address? Base=0x4000, so physical addr=0x4240

Fetch instruction at 0x4240. Get "la \$a0, varx"

Move 0x4050 → \$a0, Move PC+4→PC

2. Fetch 0x0244. Translated to Physical=0x4244. Get "jal strlen"

Move 0x0248 → \$ra (return address!), Move 0x0360 → PC

Example of segment translation (16bit address)

```
0x0240  main:    la $a0, varx
0x0244                jal strlen
...
0x0360  strlen: li  $v0, 0 ;count
0x0364  loop:   lb  $t0, ($a0)
0x0368                beq $r0,$t0, done
...
0x4050  varx    dw   0x314159
```

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

Let's simulate a bit of this code to see what happens (PC=0x0240):

1. Fetch 0x0240 (0000 0010 0100 0000).
Virtual segment #? 0; Offset? 0x240
Physical address? Base=0x4000, so physical addr=0x4240
Fetch instruction at 0x4240. Get "la \$a0, varx"
Move 0x4050 → \$a0, Move PC+4→PC
2. Fetch 0x0244. Translated to Physical=0x4244. Get "jal strlen"
Move 0x0248 → \$ra (return address!), Move 0x0360 → PC
3. Fetch 0x0360. Translated to Physical=0x4360. Get "li \$v0, 0"
Move 0x0000 → \$v0, Move PC+4→PC

Example of segment translation (16bit address)

```
0x0240  main:    la $a0, varx
0x0244                jal strlen
...
0x0360  strlen: li    $v0, 0 :count
0x0364  loop:   lb    $t0, ($a0)
0x0368                beq $r0,$t0, done
...
0x4050  varx    dw    0x314159
```

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000

Let's simulate a bit of this code to see what happens (PC=0x0240):

4. Fetch 0x0364. Translated to Physical=0x4364. Get "lb \$t0, (\$a0)"

Since \$a0 is 0x4050, try to load byte from 0x4050

Translate 0x4050 (0100 0000 0101 0000).

Virtual segment #? 1; Offset? 0x50

Physical address? Base=0x4800, Physical addr = 0x4850,

Load Byte from 0x4850→\$t0, Move PC+4→PC

Observations about segmentation

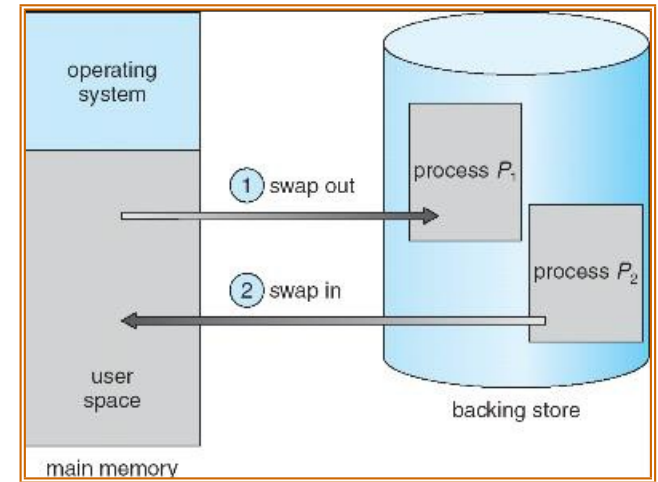
- Translation on every instruction fetch, load or store
- Virtual address space has holes
 - Segmentation efficient for sparse address spaces
- When it is OK to address outside valid range?
 - This is how the stack (and heap?) allowed to grow
 - For instance, stack takes fault, system automatically increases size of stack

Observations about segmentation

- Need protection mode in segment table
 - For example, code segment would be read-only
 - Data and stack would be read-write (stores allowed)
- What must be saved/restored on context switch?
 - Segment table stored in CPU, not in memory (small)
 - Might store all of processes memory onto disk when switched (called “swapping”)

What if not all segments fit in memory?

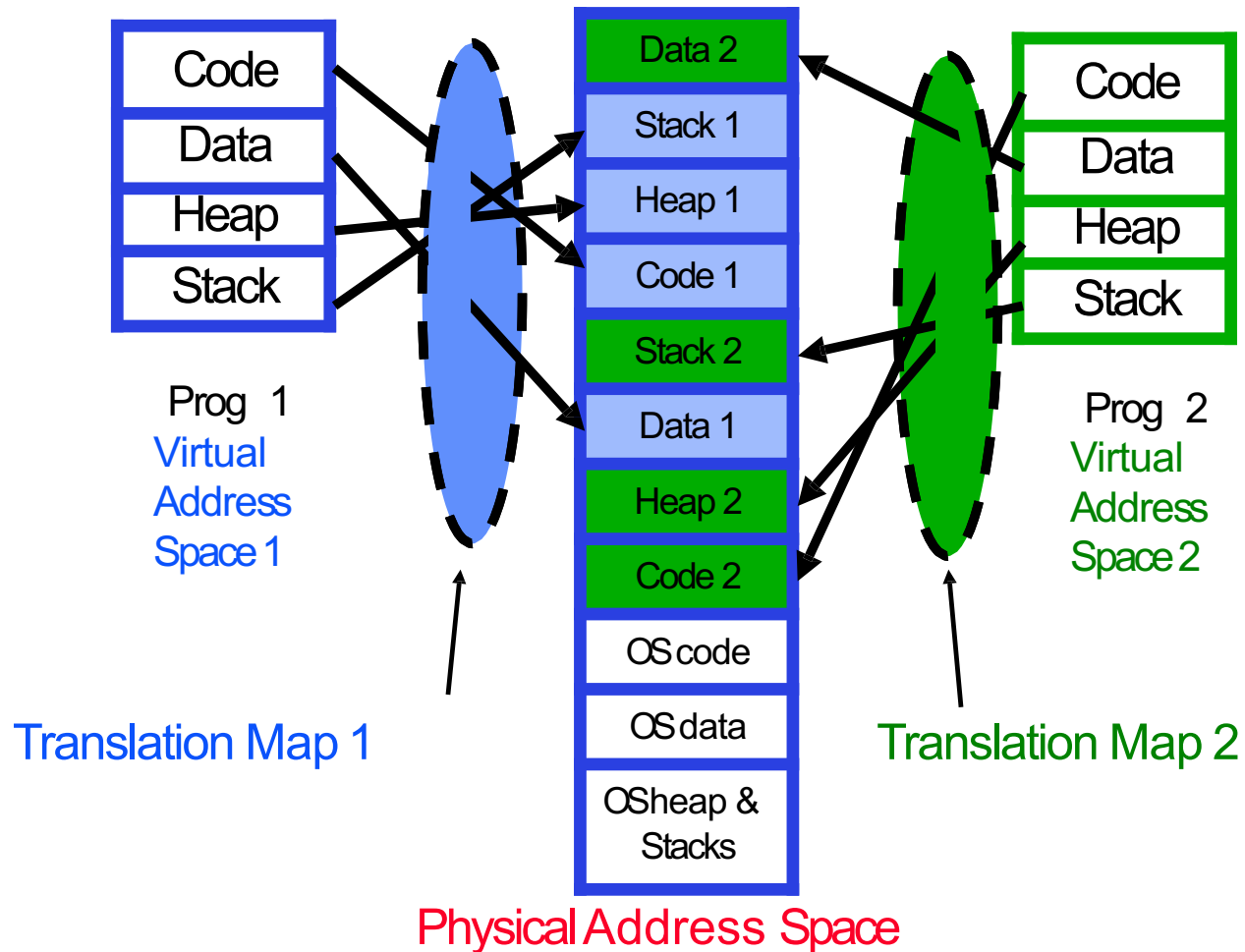
- Extreme form of context switch: **swapping**
 - To make room for next process, some or all of the previous process is moved to disk
 - Likely need to send out complete segments
 - This greatly increases the cost of context-switching
- What might be a desirable alternative?
 - **Some way to keep only active portions of a process in memory at any one time**
 - Need finer granularity control over physical memory



Problems with segmentation

- Must fit variable-sized chunks into physical memory
- May move processes multiple times to fit everything
- Limited options for swapping to disk
- **Fragmentation**: wasted space
 - **External**: free gaps between allocated chunks
 - **Internal**: don't need all memory within allocated chunks

An example with general address translation

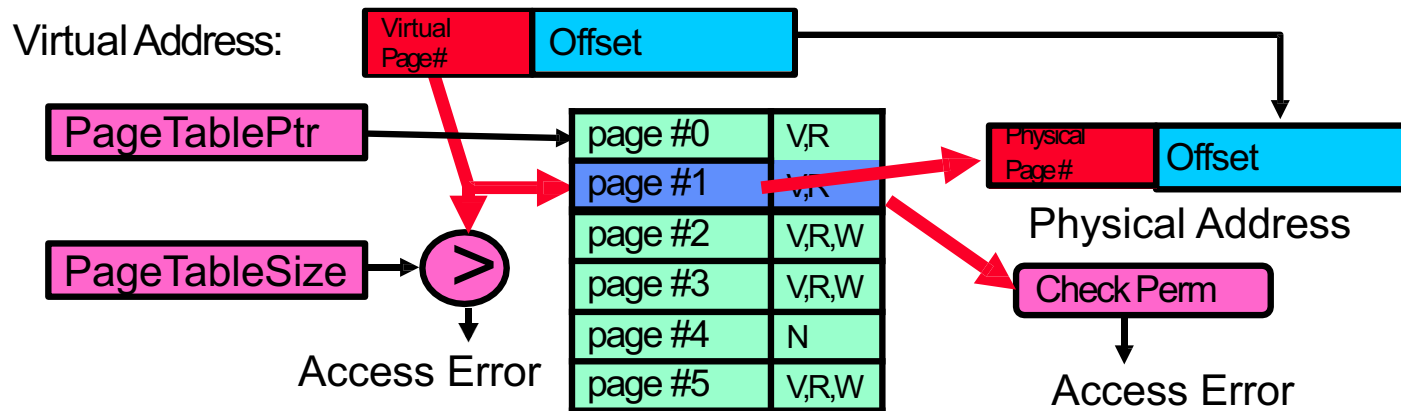


Paging

Paging: physical memory in fixed size chunks

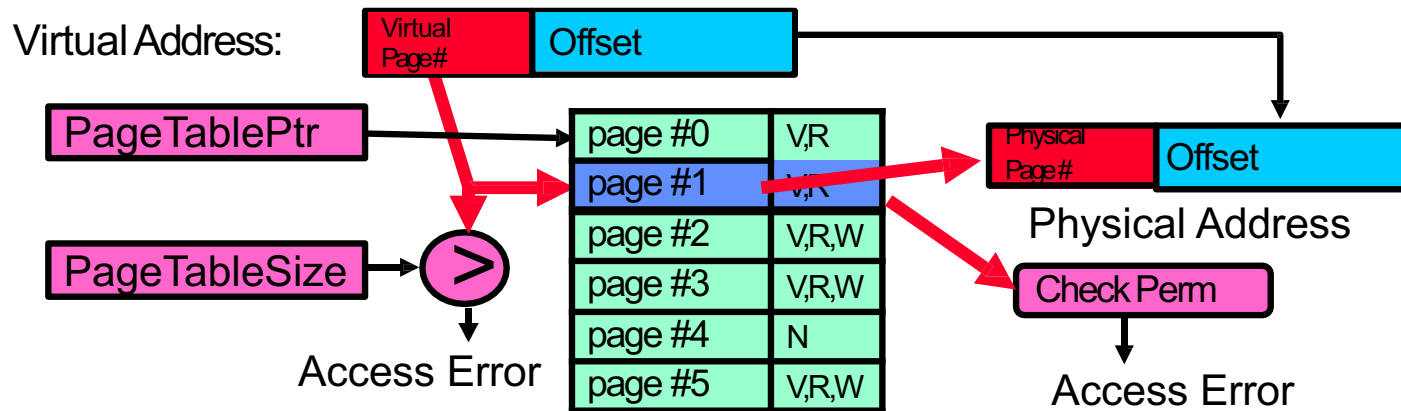
- Solution to fragmentation from segments?
 - Allocate physical memory in **fixed size** chunks (“**pages**”)
 - Every chunk of physical memory is equivalent
 - Can use simple vector of bits to handle allocation:
00110001110001101 ... 110010
 - Each bit represents page of physical memory
1 $\Rightarrow$ allocated, **0** $\Rightarrow$ free
- Should pages be as big as our previous segments?
 - No: Can lead to lots of internal fragmentation
 - Typically have small pages (1K-16K)
 - Consequently: need multiple pages/segment

How to implement simple paging?



- Page Table (one per process)
 - Resides in physical memory
 - Contains physical page and permission for each virtual page (e.g. Valid bits, Read, Write, etc)

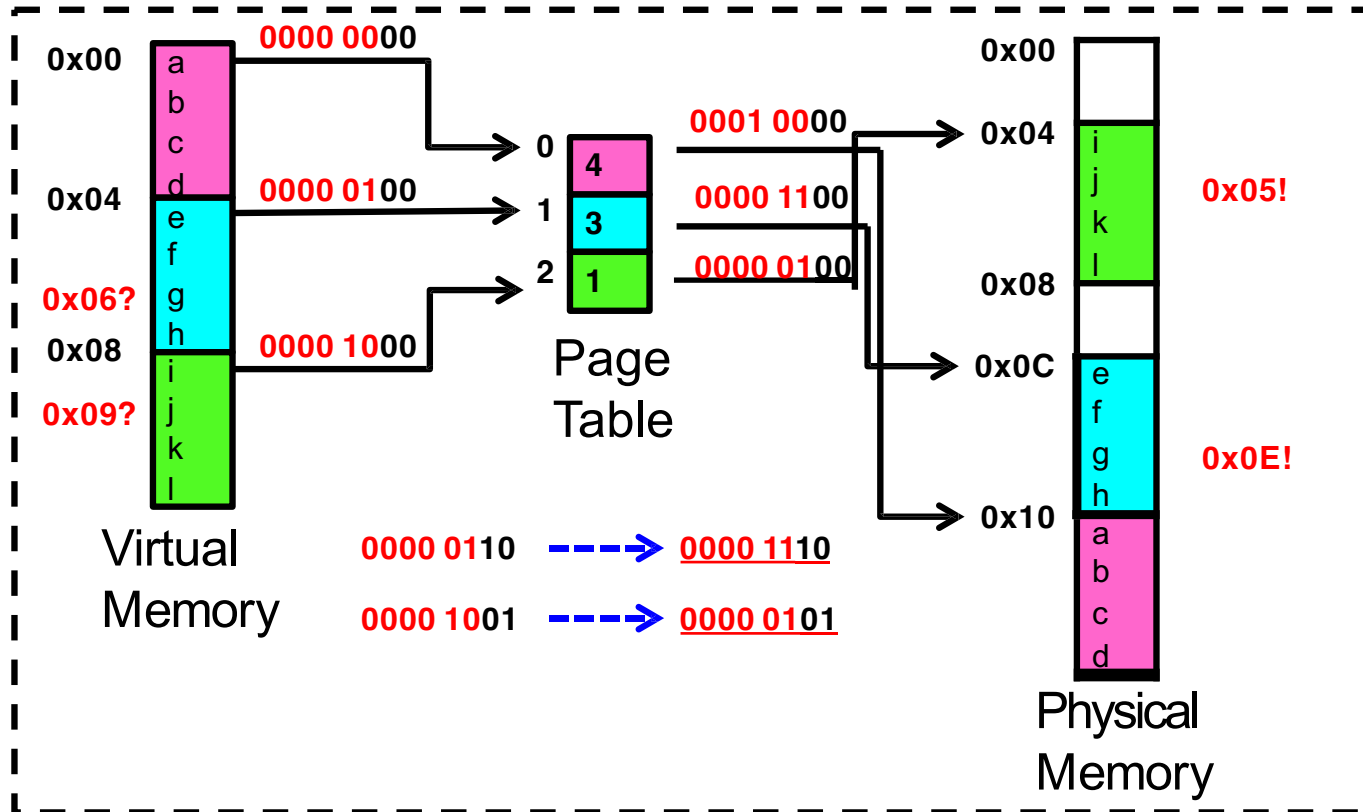
How to implement simple paging?



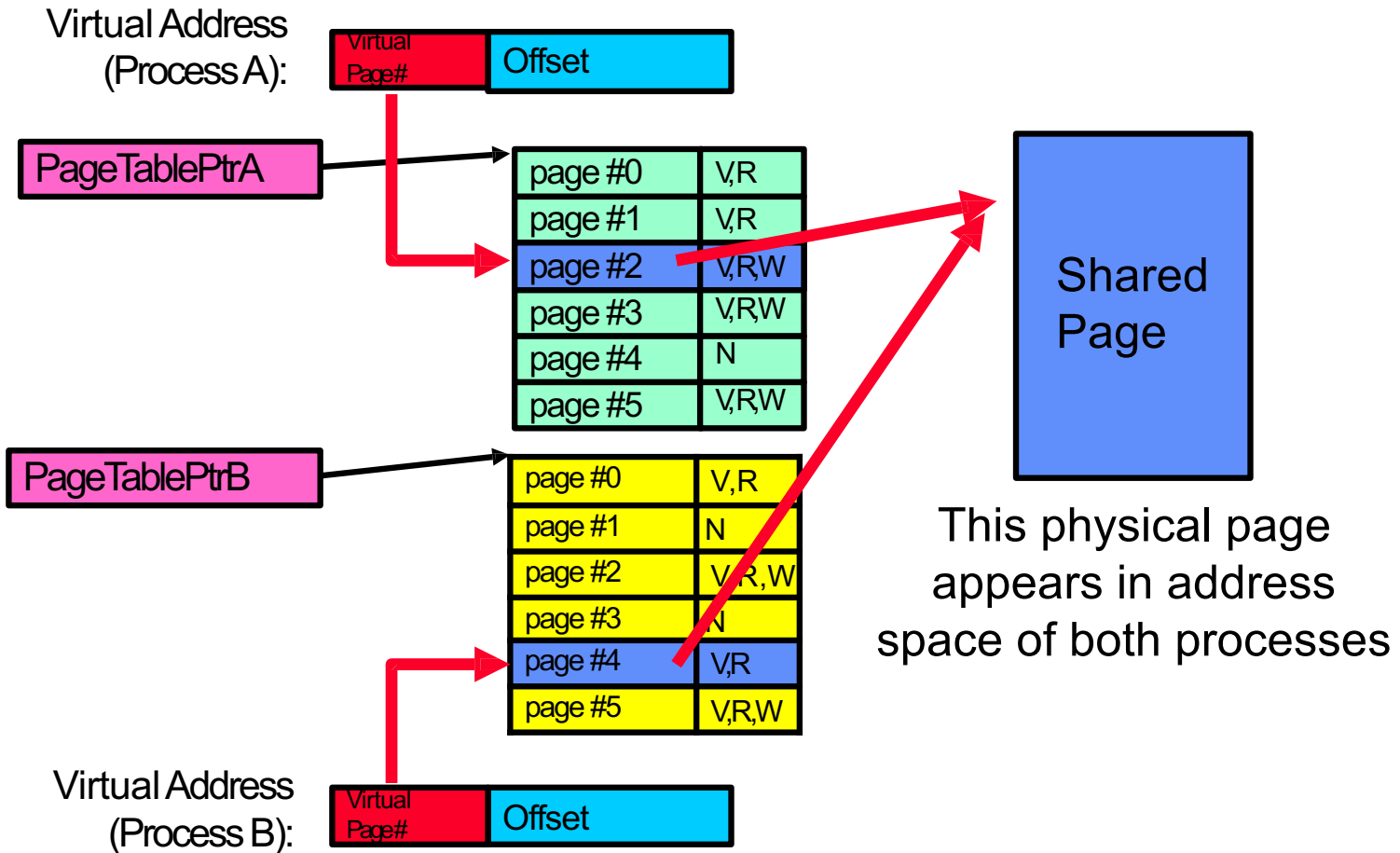
- Virtual address mapping
 - Offset from Virtual address copied to Physical Address
 - Example: 10 bit offset $\Rightarrow$ 1024-byte pages
 - Virtual page # is all remaining bits
 - Example for 32-bits: $32 - 10 = 22$ bits, i.e. 4 million entries
 - Physical page # copied from table into physical address
- Check Page Table bounds and permissions

Simple page table example

Example (4 byte pages)



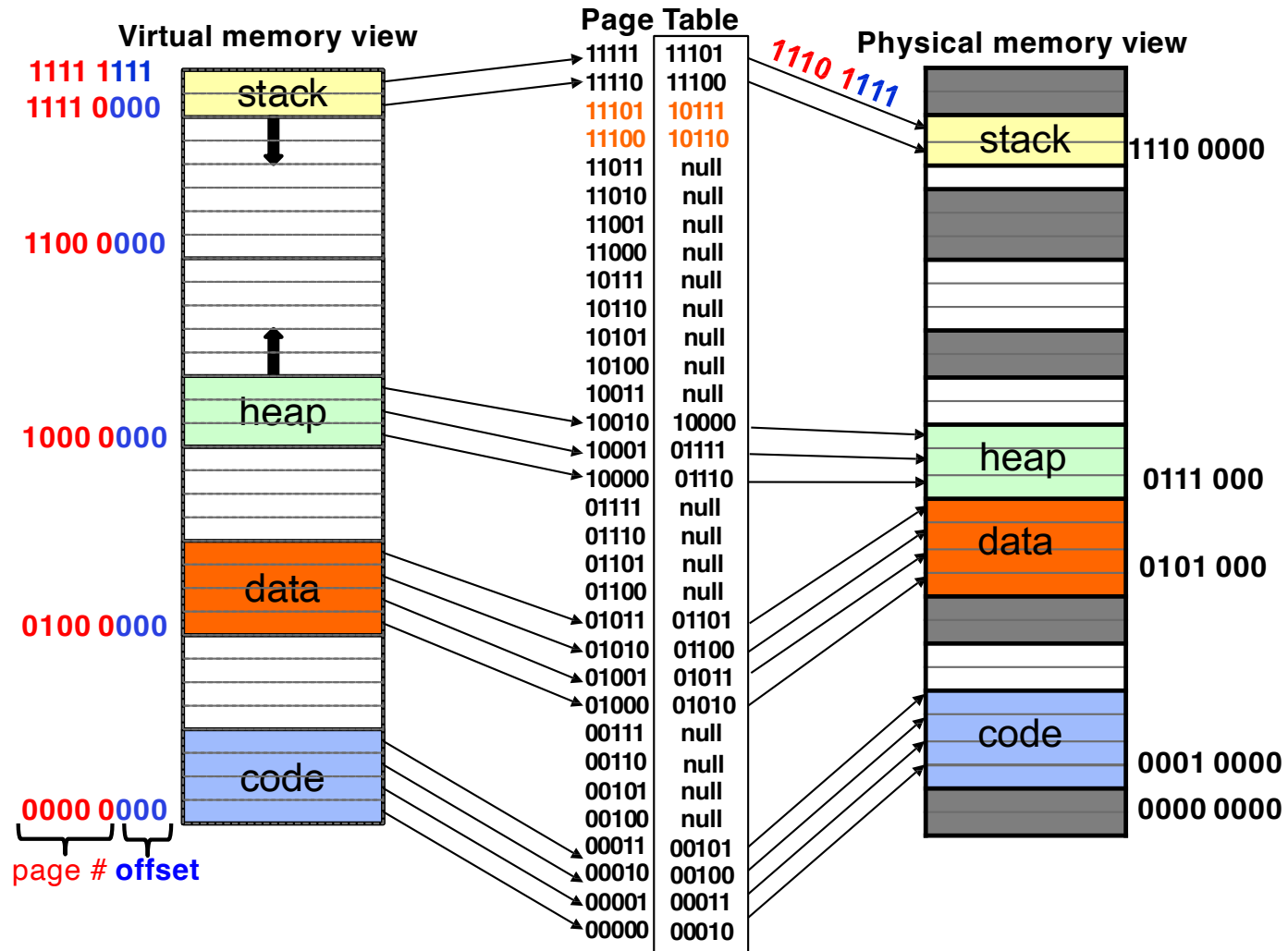
What about sharing?



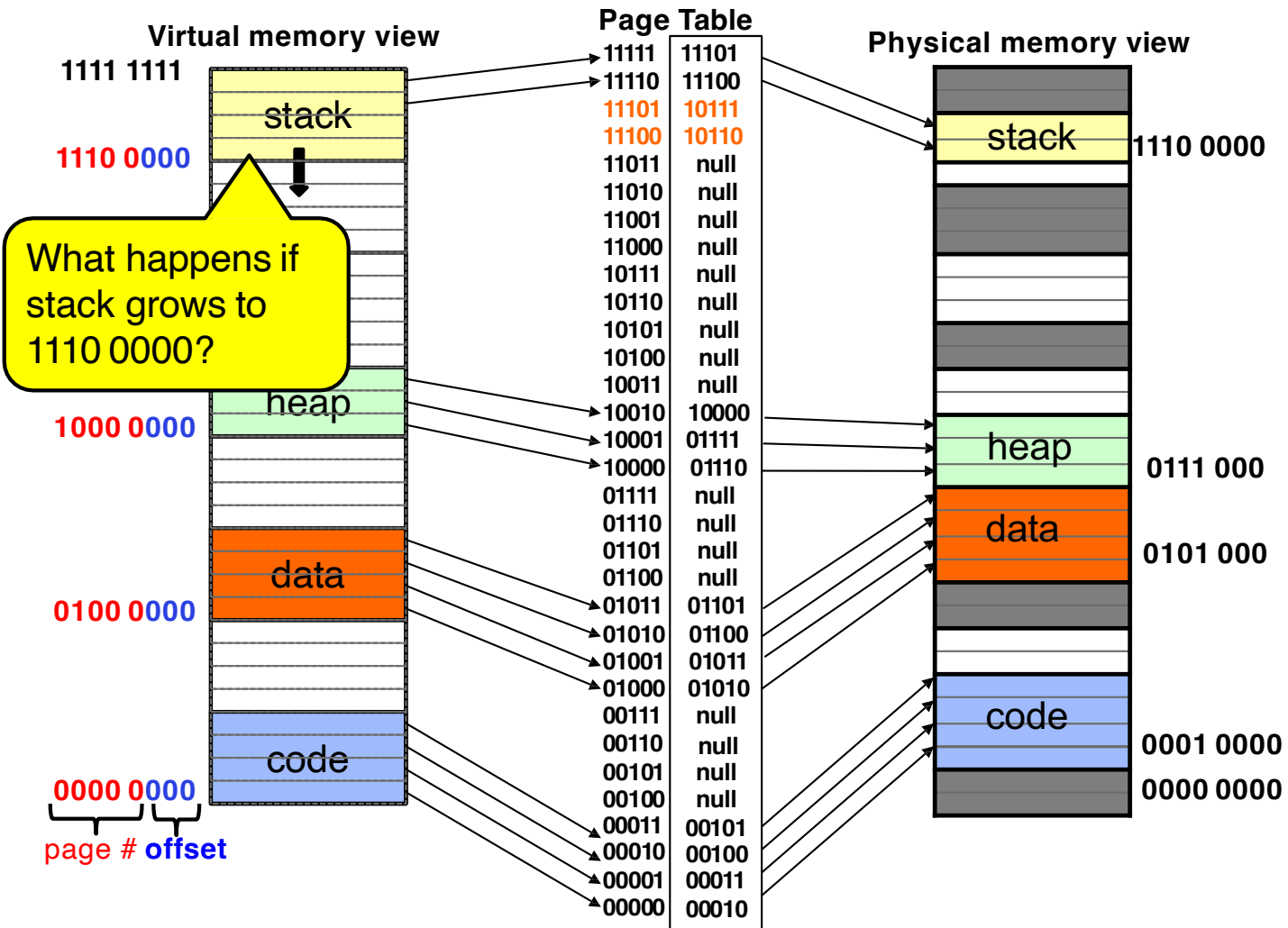
Where is page sharing used?

- The “kernel region” of every process has the same page table entries
 - The process cannot access it at user level
 - But on U→K switch, kernel code can access it AS WELL AS the region for THIS user
 - What does the kernel need to do to access other user processes?
- Different processes running same binary!
 - Execute-only, but do not need to duplicate code segments
- User-level system libraries (execute only)
- Shared-memory segments between different processes
 - Can actually shared objects directly between processes
 - Must map page into same place in address space!
 - This is a limited form of the sharing that threads have within a single process

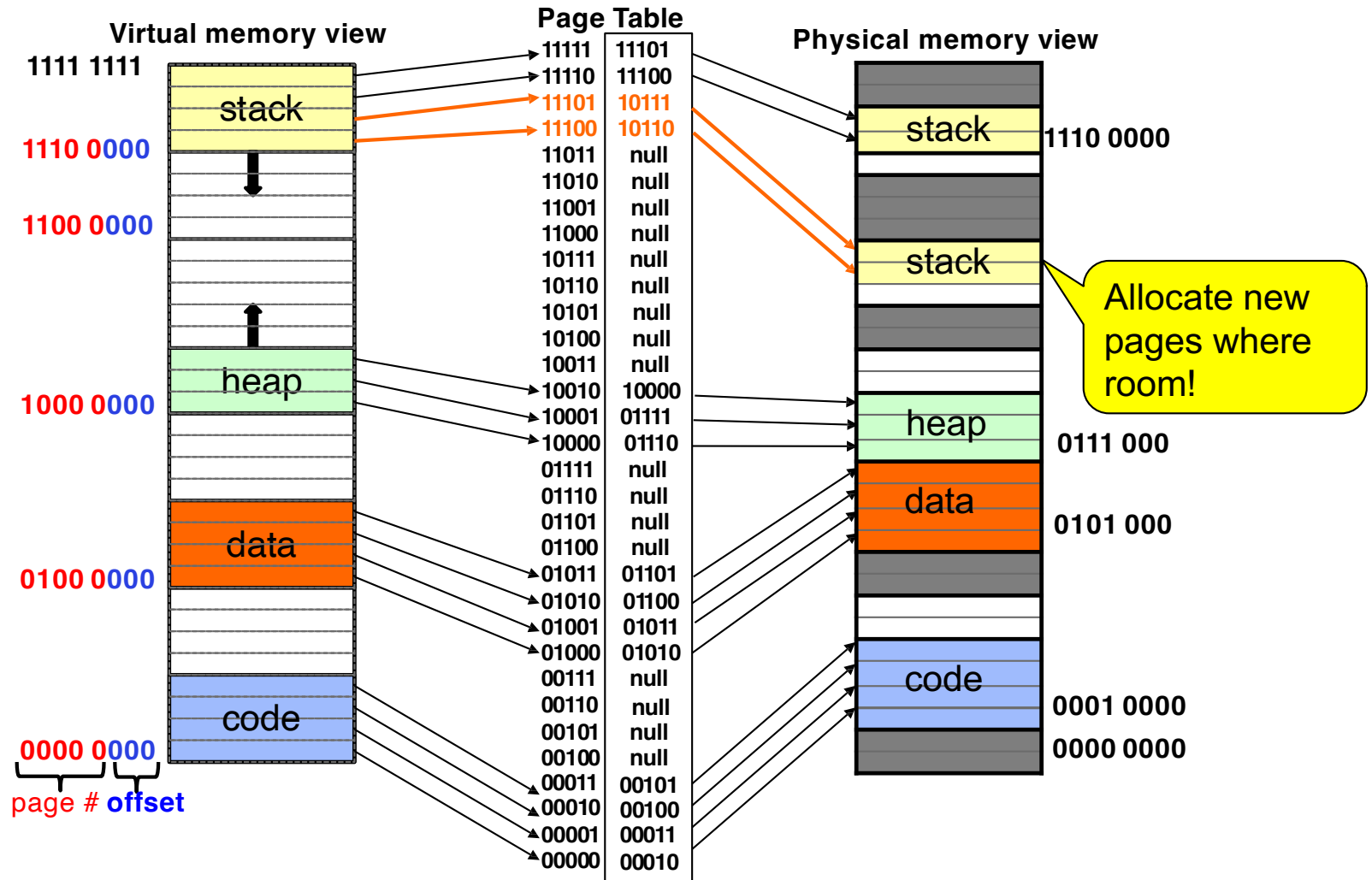
Summary: paging



Summary: paging



Summary: paging



Conclusion

- Segment mapping
 - Segment registers within processor
 - Segment ID associated with each access
 - Often comes from portion of virtual address
 - Can come from bits in instruction instead (x86)
 - Each segment contains base and limit information
 - Offset (rest of address) adjusted by adding base

Conclusion

- Page tables
 - Memory divided into fixed-sized chunks of memory
 - Virtual page number from virtual address mapped through page table to physical page number
 - Offset of virtual address same as physical address
 - Large page tables can be placed into virtual memory
- Next time: multi-level tables
 - Virtual address mapped to series of tables
 - Permit sparse population of address space

Reading for next lecture

- Operating System Concepts:
 - Chapter 9.1-9.5

Or

- Operating System: Three Easy Pieces:
 - Chapters 12-19